

Indice

Introduzione	3
Analisi di una soluzione parallela MPI e una OMP, problema da risolvere	3
Informazioni sistema	4
Analisi delle prestazioni della soluzione parallela MPI	5
Scalabilità strong MPI – Sistema di test.....	5
Scalabilità strong MPI – Dati sperimentali	5
Scalabilità Strong MPI – Speedup	5
Scalabilità strong MPI – Efficienza	6
Scalabilità strong MPI – Conclusioni	6
Scalabilità weak MPI – Sistema di test	7
Scalabilità weak MPI – Dati sperimentali.....	7
Scalabilità weak MPI – Scaled speedup	7
Scalabilità weak MPI – Efficienza.....	8
Scalabilità weak MPI – Conclusioni	8
Analisi delle prestazioni della soluzione parallela OMP	9
Scalabilità strong OMP – Sistema di test	9
Scalabilità strong OMP – Dati sperimentali	9
Scalabilità strong OMP – Speedup.....	9
Scalabilità strong OMP – Efficienza	10
Scalabilità strong OMP – Conclusioni	10
Scalabilità weak OMP – Sistema di test.....	11
Scalabilità weak OMP – Dati sperimentali	11
Scalabilità weak OMP – Scaled speedup.....	11
Scalabilità weak OMP – Efficienza	12
Scalabilità weak OMP – Conclusioni.....	12
Grafici di sintesi	13
Scalabilità strong.....	13
Scalabilità weak	13
Dati raw	14
Altre esecuzioni effettuate	16

Codice C per risolvere i due problemi	18
Soluzione MPI.....	18
Soluzione OpenMP	22
Script usati per automatizzare il lancio dei test	24
Launcher mpi scalabilità strong	24
Launcher mpi scalabilità weak.....	25
Launcher omp scalabilità strong	26
Launcher omp scalabilità weak.....	27

Introduzione

Analisi di una soluzione parallela MPI e una OMP, problema da risolvere

Sia data una matrice quadrata $A[N][N]$, contenente N^2 valori reali.

Si supponga che la matrice A sia molto grande ($N \geq 2000$).

Si vuole trasformare la matrice A in una matrice di interi binaria $T[N][N]$ nella quale ogni elemento può assumere solo 1 valore appartenente al dominio $\{0,1\}$.

In particolare, progettare un programma che calcoli la matrice binaria $T[N][N]$ di valori interi come segue.

per ogni $a_{ij} \in A, i \in 0, N-1, j \in 0, N-1$ si consideri il suo intorno lij .

Per calcolare T , data la matrice A :

- per ogni $a_{ij} \in A, i \in 0, N-1, j \in 0, N-1$ si consideri il suo intorno $A_{ij}[3][3]$
- Sia m_{ij} la media aritmetica dei valori appartenenti all'intorno A_{ij} :

Ogni $t_{ij} \in T, i \in 0, N-1, j \in 0, N-1$ avrà un valore calcolato con il seguente criterio:

- se $a_{ij} > m_{ij} \rightarrow t_{ij} = 1$
- se $a_{ij} \leq m_{ij} \rightarrow t_{ij} = 0$

Informazioni sistema

Nome: GALILEO100 Cluster / Linux Infiniband Cluster

OS: CentOS 8.3

Nodi: 554 compute nodes with 2 x CPU Intel CascadeLake 8260(24 cores, 2.4 GHz)

Ram(nodo): 384GB RAM DDR4

Tipi di nodi:

- 340 standard nodes ("thin nodes") 480 GB SSD
- 180 data processing nodes ("fat nodes") 2TB SSD, 3TB Intel Optane
- a34 GPU nodes (visualization "viz") with 2x NVIDIA GPU V100

Rete inter-nodo: Mellanox Infiniband HDR100

Workload Manager: SLURM 23.11

Analisi delle prestazioni della soluzione parallela MPI

Scalabilità strong MPI – Sistema di test

Per testare la scalabilità Strong bisogna aumentare il numero di processi mantenendo costante la dimensione del problema. Per fare questo con un susseguirsi di esecuzioni sequenziali del programma pilotate da uno script sbatch aumento ad ogni esecuzione il numero di **processi** mantenendo la dimensione del problema N invariata.

Scalabilità strong MPI – Dati sperimentali

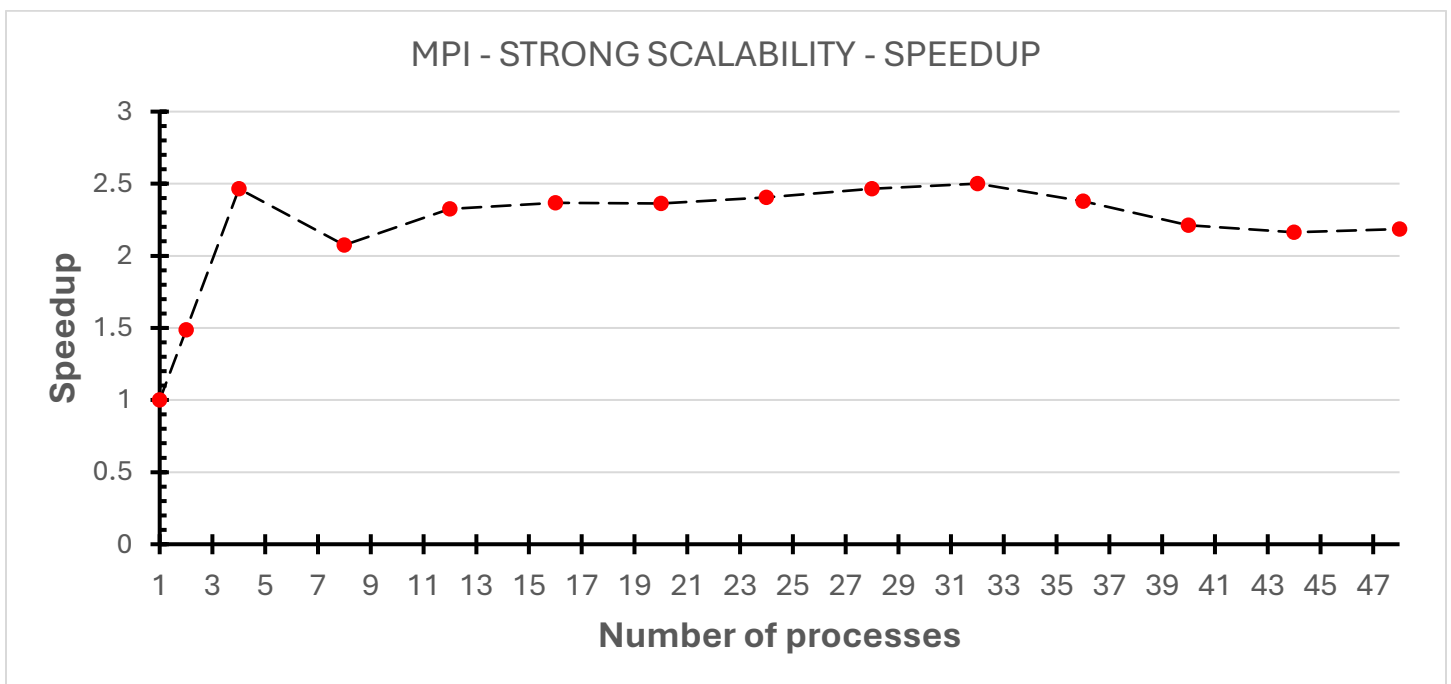
P	N	Tempo (s)
1	20000	4.678177
2	20000	3.145589
4	20000	1.89778
8	20000	2.254403
12	20000	2.011689
16	20000	1.976858
20	20000	1.980196
24	20000	1.944755
28	20000	1.898041
32	20000	1.870832
36	20000	1.966869
40	20000	2.114263
44	20000	2.162107
48	20000	2.140852

P = numero di processi

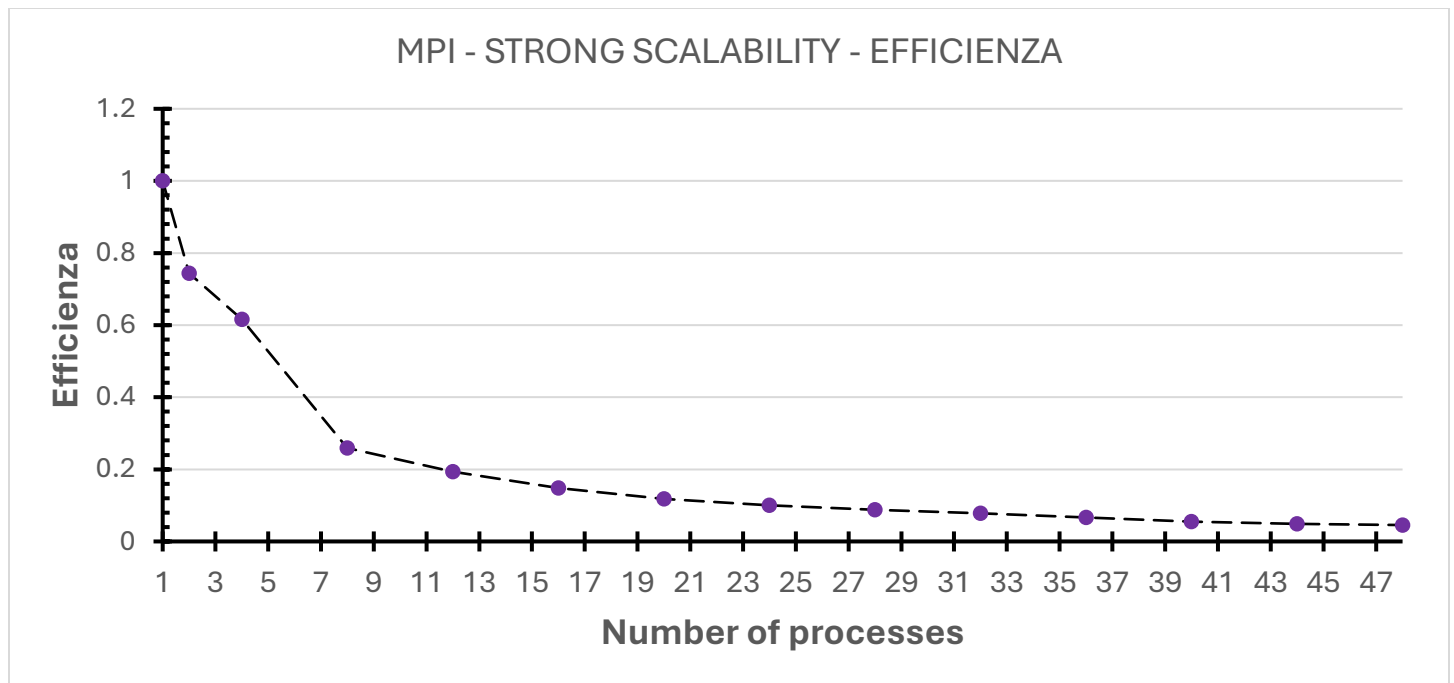
N = numero di righe della matrice quadrata

Tempo = tempo di esecuzione del codice per risolvere il problema (secondi)

Scalabilità Strong MPI – Speedup



Scalabilità strong MPI – Efficienza



Scalabilità strong MPI – Conclusioni

Dai dati ottenuti dai test per la scalabilità strong per l'algoritmo basato sul modello a scambio di messaggi si riesce a notare uno speedup limitato e una saturazione molto veloce. L'efficienza è accettabile solo per i primi processi e decresce velocemente facendo intuire la non convenienza nell'aumentare il numero di processi oltre a una certa soglia. Questo indica che il programma **non scala in modo ideale** per la scalabilità forte a causa del grande overhead per la comunicazione tra processi.

Scalabilità weak MPI – Sistema di test

Per testare la scalabilità weak bisogna aumentare il numero di processi mantenendo costante la dimensione del problema per processo, quindi, nel nostro caso bisogna mantenere il numero di valori per task costante. Per fare questo con un susseguirsi di esecuzioni sequenziali del programma pilotate da uno script sbatch aumento ad ogni esecuzione il numero di **processi** e adatto, di conseguenza, la dimensione del problema.

Scalabilità weak MPI – Dati sperimentali

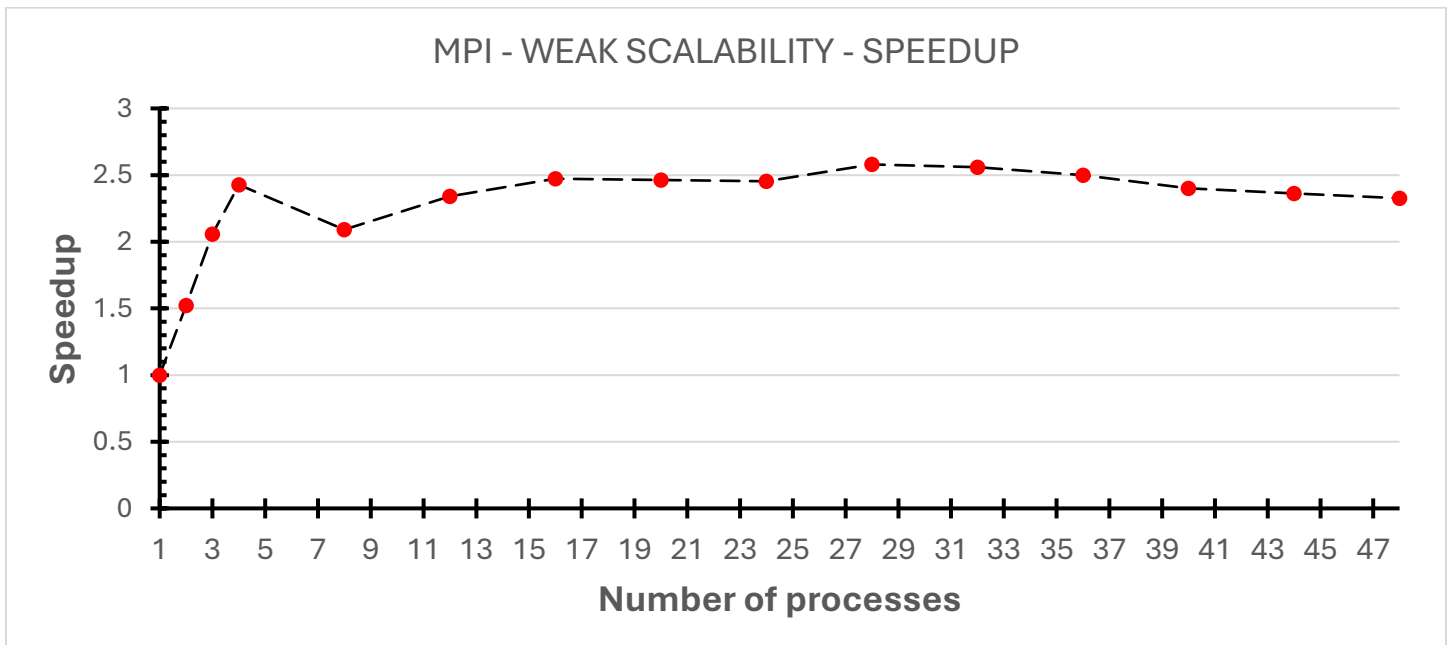
P	N	Tempo (s)
1	2000	0.04962
2	2828	0.065219
3	3464	0.072371
4	4000	0.081802
8	5657	0.189768
12	6928	0.254503
16	8000	0.321178
20	8944	0.403028
24	9798	0.485454
28	10583	0.538566
32	11314	0.620423
36	12000	0.714896
40	12649	0.826433
44	13266	0.924355
48	13856	1.023487

P = numero di processi

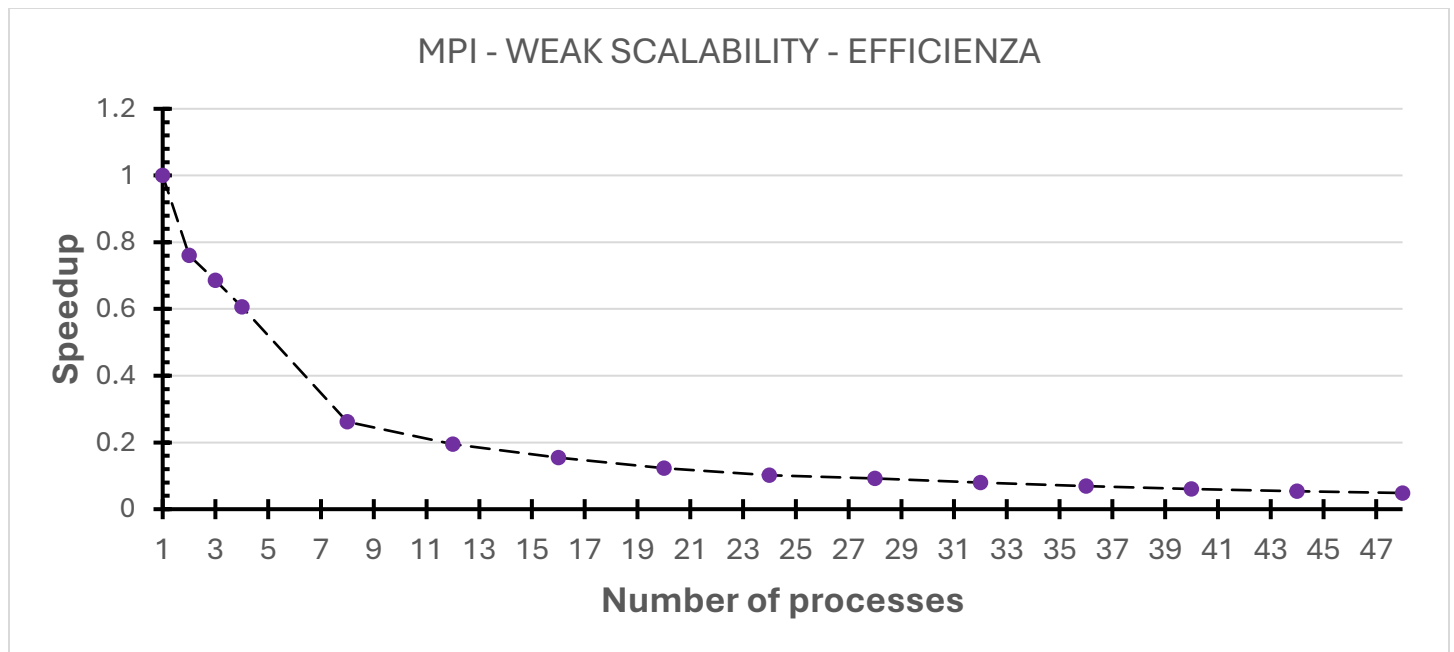
N = numero di righe della matrice quadrata

Tempo = tempo di esecuzione del codice per risolvere il problema (secondi)

Scalabilità weak MPI – Scaled speedup



Scalabilità weak MPI – Efficienza



Scalabilità weak MPI – Conclusioni

Nel caso ideale per la scalabilità weak aumentando il numero di processi e mantenendo costante la dimensione del problema per processo (nel caso in esame corrispondente al numero di valori da elaborare per ciascun processo), il tempo dovrebbe rimanere circa costante. Dai dati di questo test ci rendiamo quindi conto che anche per la scalabilità weak questa implementazione **non scala in modo ideale** perché aumentando il numero di processi anche il tempo esecuzione aumenta in modo significativo, lo speedup satura molto velocemente e l'efficienza decresce rapidamente scendendo molto velocemente sotto al 20%. Quindi anche in questo caso i costi di comunicazione, la sincronizzazione e la gestione dei dati fanno in modo che questa implementazione non abbia una buona weak scalability.

Analisi delle prestazioni della soluzione parallela OMP

Scalabilità strong OMP – Sistema di test

Per testare la scalabilità strong in OpenMP(Open MultiProcessing) bisogna aumentare il numero di thread mantenendo costante la dimensione del problema. Per fare questo con un susseguirsi di esecuzioni sequenziali del programma pilotate da uno script sbatch aumento ad ogni esecuzione il numero di **thread nel nodo** mantenendo la dimensione del problema N invariata.

Scalabilità strong OMP – Dati sperimentali

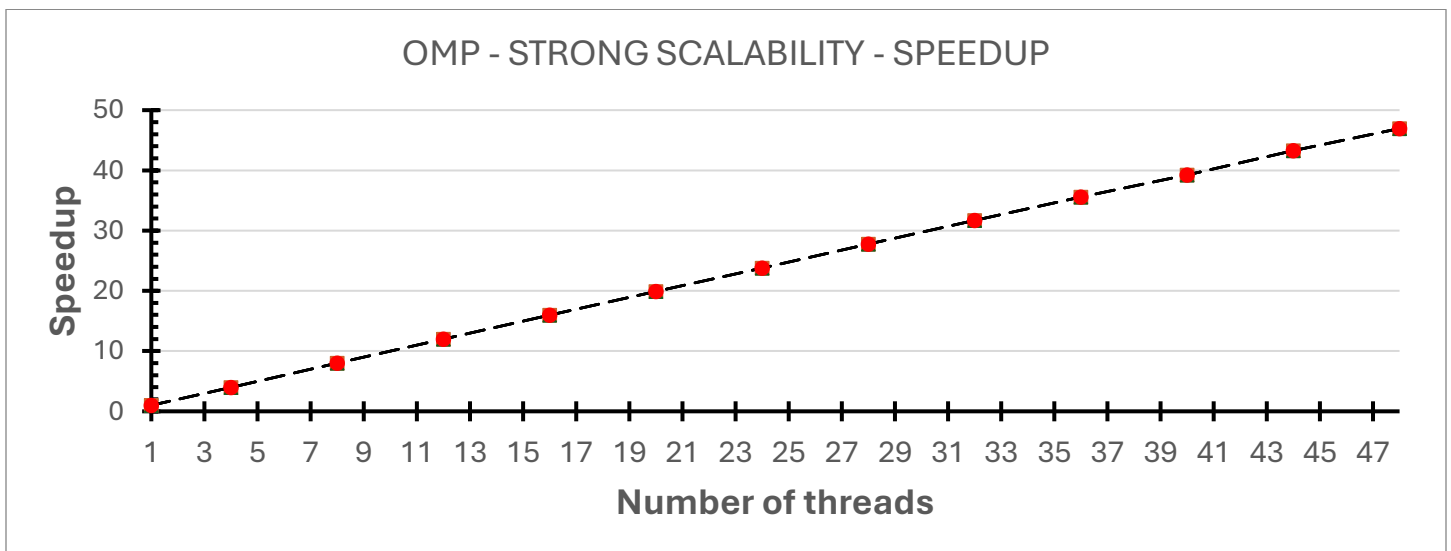
P	N	Tempo (s)
1	20000	19.97955
4	20000	5.008331
8	20000	2.503268
12	20000	1.669482
16	20000	1.252983
20	20000	1.003598
24	20000	0.839344
28	20000	0.719843
32	20000	0.630489
36	20000	0.561788
40	20000	0.509048
44	20000	0.46155
48	20000	0.425582

P = numero di processi

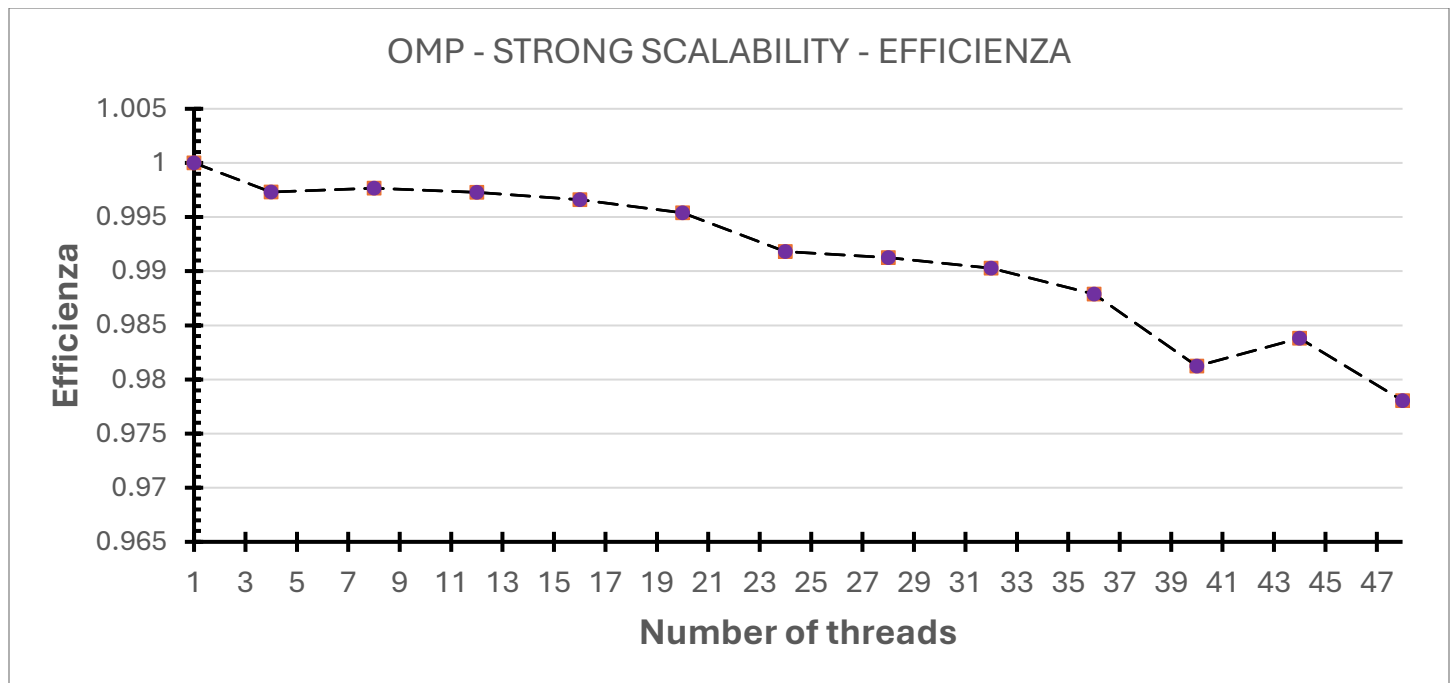
N = numero di righe della matrice quadrata

Tempo = tempo di esecuzione del codice per risolvere il problema (secondi)

Scalabilità strong OMP – Speedup



Scalabilità strong OMP – Efficienza



Scalabilità strong OMP – Conclusioni

Per la scalabilità strong per l'implementazione OpenMP dai test risulta una **scalabilità praticamente ideale**. Lo speedup cresce linearmente con il numero dei thread e l'efficienza rimane sempre approssimabile a 1. Si nota una leggera tendenza dell'efficienza a diminuire con il numero dei thread e quindi ipotizzando di poter aumentare ancora il numero di processi probabilmente sul lungo andare si osserverà una tendenza a saturare dello speedup secondo la legge di Amdahl. Questa leggera diminuzione delle prestazioni sul lungo andare potrebbe essere dovuta a una parte di codice sequenziale o all'aumento della competizione per l'accesso alla memoria.

Scalabilità weak OMP – Sistema di test

Per testare la scalabilità weak bisogna aumentare il numero di thread mantenendo costante la dimensione del problema per thread, quindi, nel nostro caso bisogna mantenere il numero di valori per thread costante. Per fare questo con un susseguirsi di esecuzioni sequenziali del programma pilotate da uno script sbatch aumento ad ogni esecuzione il numero di **thread** e adatto, di conseguenza, la dimensione del problema.

Scalabilità weak OMP – Dati sperimentali

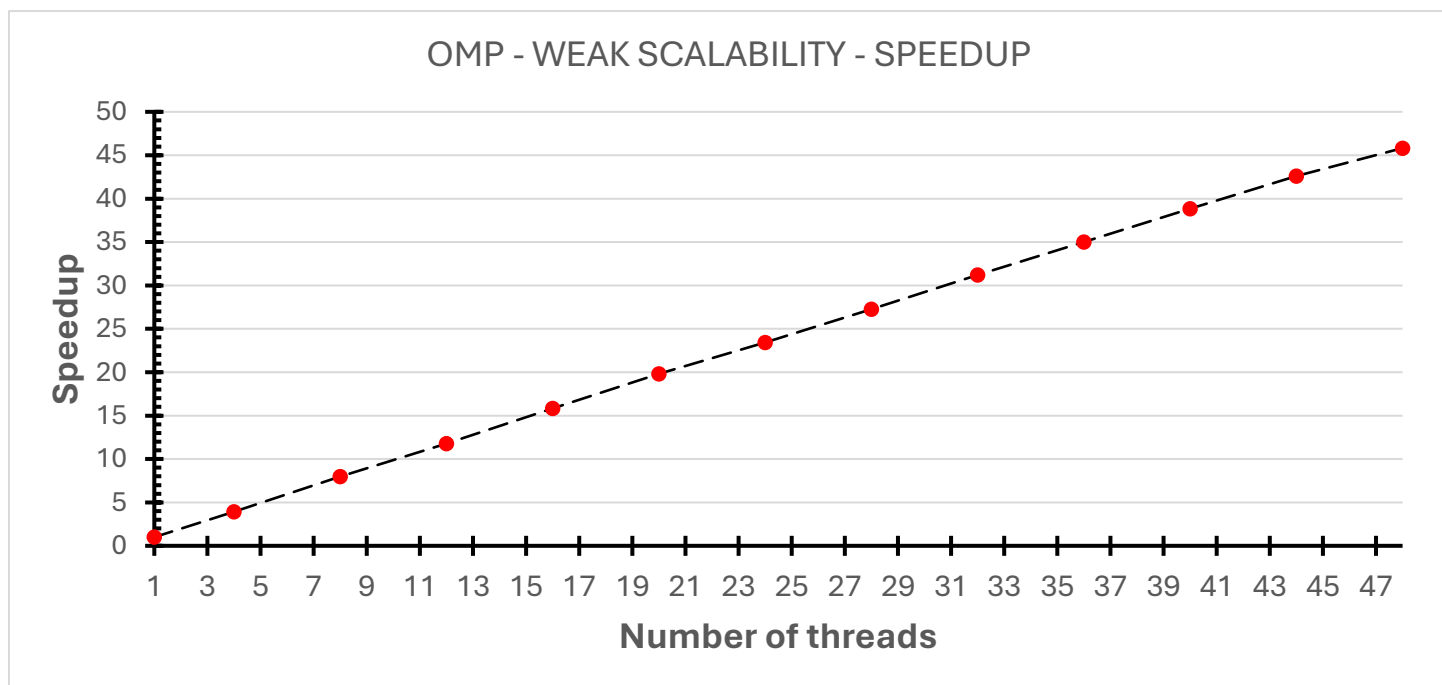
P	N	Tempo (s)
1	2000	0.186552
4	4000	0.189878
8	5657	0.187418
12	6928	0.190065
16	8000	0.188325
20	8944	0.188221
24	9798	0.191021
28	10583	0.191592
32	11314	0.191323
36	12000	0.191788
40	12649	0.192055
44	13266	0.192686
48	13856	0.195418

P = numero di processi

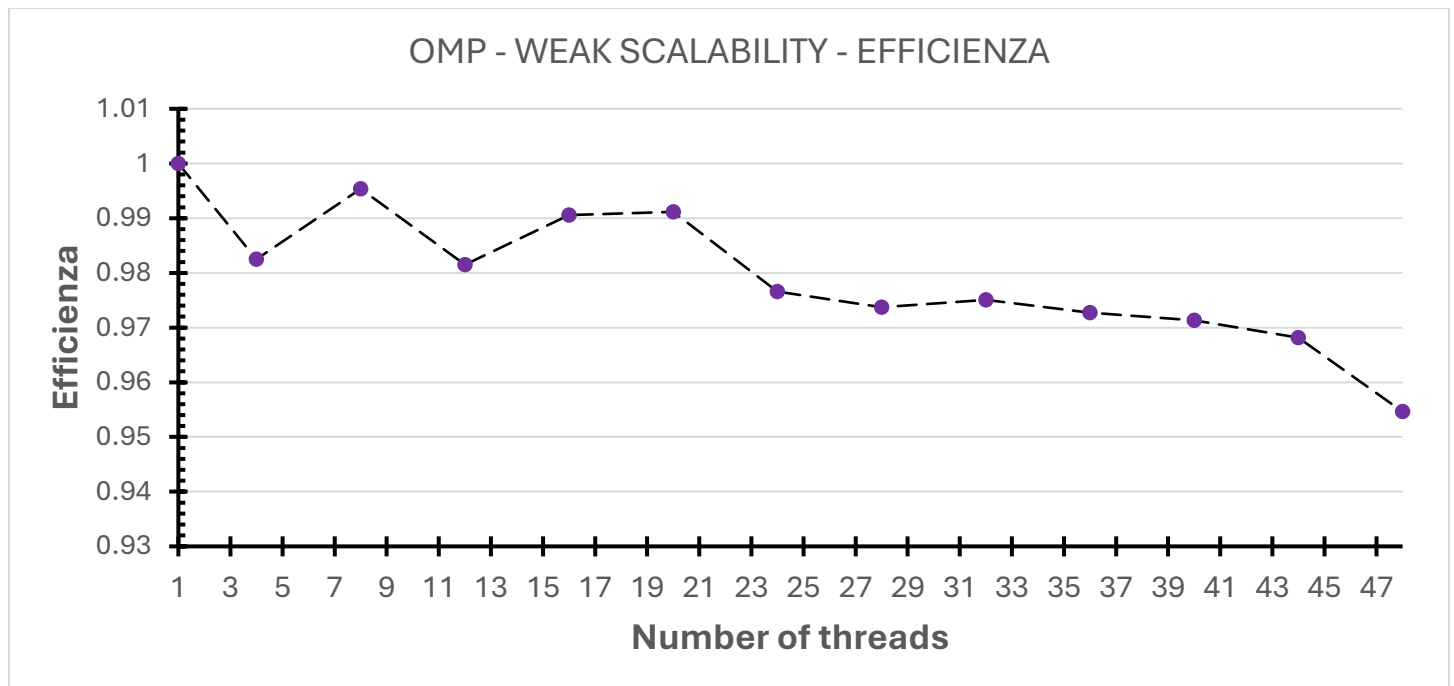
N = numero di righe della matrice quadrata

Tempo = tempo di esecuzione del codice per risolvere il problema (secondi)

Scalabilità weak OMP – Scaled speedup



Scalabilità weak OMP – Efficienza

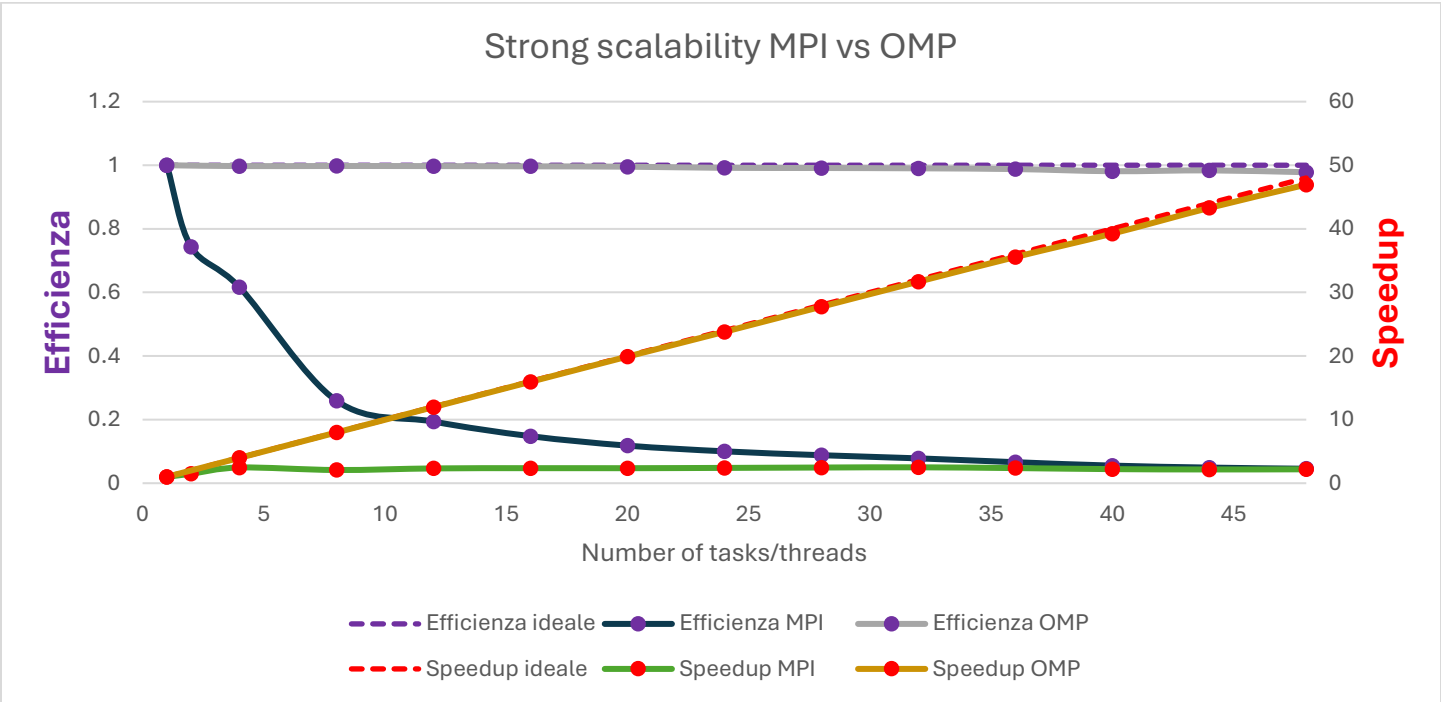


Scalabilità weak OMP – Conclusioni

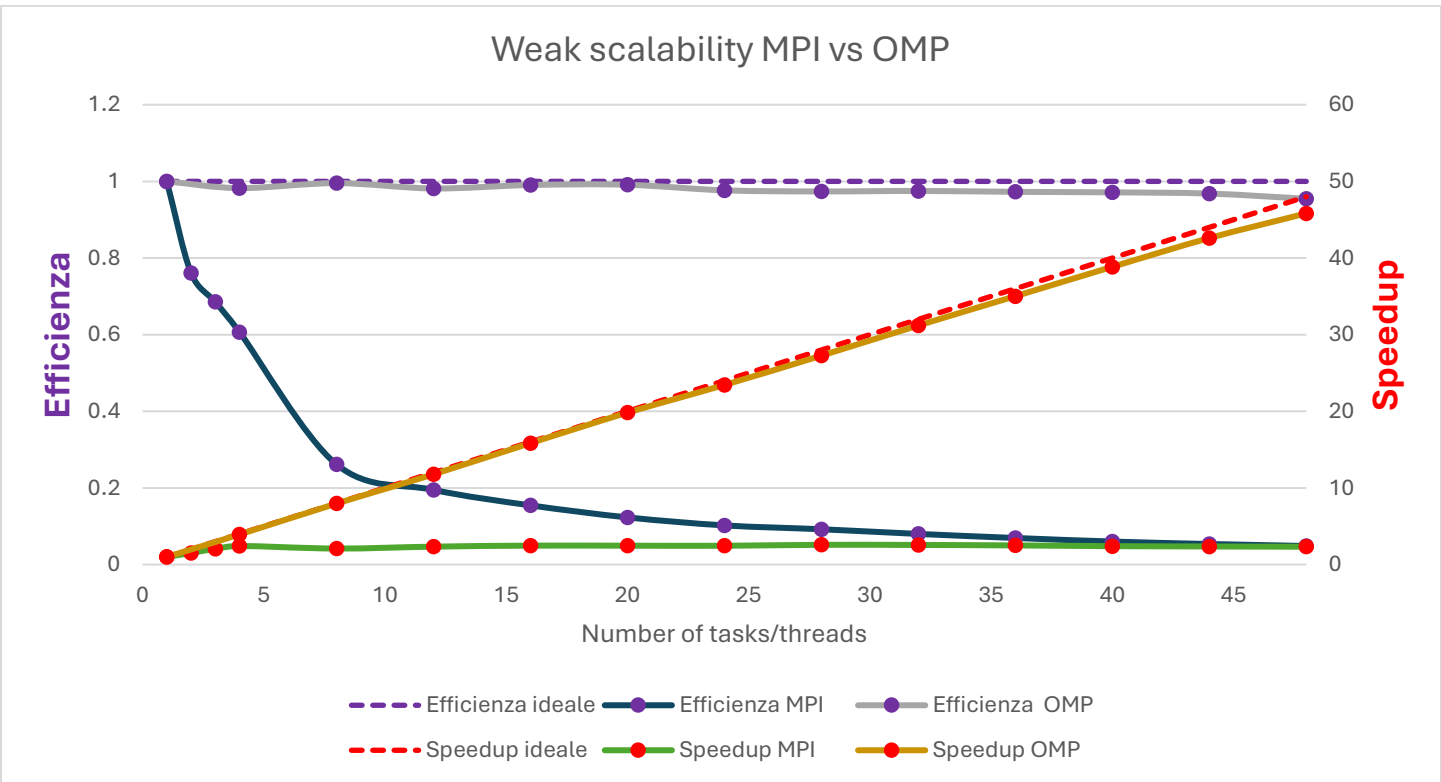
Dai test effettuati sull'implementazione OMP per la scalabilità weak si evidenzia come per la scalabilità strong un **eccellente capacità di scalare** considerabile nella pratica quasi ideale. Il grafico dello Speedup mostra una crescita lineare indicando che non ci sono parti non parallelizzabile che rappresentano un significativo bottleneck (Legge di Gustafson). L'efficienza come per la scalabilità strong rimane molto alta (sempre $\geq 95\%$), il leggero calo di prestazioni che si nota con l'aumentare dei thread è normale e potrebbe essere dovuto a una parte di codice non parallelizzabile o all'aumento della competizione per l'accesso alla memoria.

Grafici di sintesi

Scalabilità strong



Scalabilità weak



Dati raw

Legenda:

P=numero di processi / N=numero di righe della matrice quadrata
sequentialT=tempo di esecuzione parte sequenziale(secondi) (generazione matrice A)

parallelT=tempo di esecuzione totale(secondi) della parte parallela, (generazione matrice T)

TotalT=tempo di esecuzione totale(secondi) / Nodes=numero di nodi

1. Scalabilità strong MPI

STRONG SCALABILITY MPI							
P	N	sequentialT(s)	parallelT(s)	totalT(s)	nodes	Speedup	Efficienza
1	20000	7.743335	4.678177	12.421512	1	1	1
2	20000	7.797794	3.145589	10.943383	1	1.487218133	0.743609067
4	20000	7.713643	1.89778	9.611423	1	2.465078671	0.616269668
8	20000	7.748253	2.254403	10.002656	1	2.075128981	0.259391123
12	20000	8.004817	2.011689	10.016506	1	2.325497132	0.193791428
16	20000	7.759202	1.976858	9.73606	1	2.366470935	0.147904433
20	20000	7.759671	1.980196	9.739867	1	2.362481795	0.11812409
24	20000	7.759193	1.944755	9.703948	1	2.405535402	0.100230642
28	20000	7.770939	1.898041	9.66898	1	2.464739697	0.088026418
32	20000	7.751816	1.870832	9.622648	1	2.50058637	0.078143324
36	20000	7.797686	1.966869	9.764555	1	2.378489366	0.066069149
40	20000	7.905849	2.114263	10.020112	1	2.212675055	0.055316876
44	20000	8.164091	2.162107	10.326198	1	2.163712064	0.049175274
48	20000	8.451424	2.140852	10.592276	1	2.185194026	0.045524876

2. Scalabilità weak MPI

WEAK SCALABILITY MPI							
P	N	sequentialT(s)	parallelT(s)	totalT(s)	nodes	Speedup	Efficienza
1	2000	0.07754	0.04962	0.12716	1	1	1
2	2828	0.15524	0.065219	0.220459	1	1.521642466	0.760821233
3	3464	0.232235	0.072371	0.304606	1	2.056901245	0.685633748
4	4000	0.309459	0.081802	0.391261	1	2.426346544	0.606586636
8	5657	0.620892	0.189768	0.81066	1	2.091817377	0.261477172
12	6928	0.928441	0.254503	1.182944	1	2.339618786	0.194968232
16	8000	1.237602	0.321178	1.55878	1	2.471900317	0.15449377
20	8944	1.548178	0.403028	1.951206	1	2.462359935	0.123117997
24	9798	1.864579	0.485454	2.350033	1	2.453126352	0.102213598
28	10583	2.169	0.538566	2.707566	1	2.579739531	0.092133555
32	11314	2.478096	0.620423	3.098519	1	2.559286164	0.079977693
36	12000	2.784663	0.714896	3.499559	1	2.4987131	0.069408697
40	12649	3.127678	0.826433	3.954111	1	2.401646594	0.060041165
44	13266	3.463782	0.924355	4.388137	1	2.361949684	0.053680675
48	13856	3.925359	1.023487	4.948846	1	2.327103324	0.048481319

3. Scalabilità strong OMP

STRONG SCALABILITY OMP							
P	N	sequentialT(s)	parallelT(s)	totalT(s)	nodes	Speedup	Efficienza
1	20000	4.144936	19.97955	24.12449	1	1	1
4	20000	4.102913	5.008331	9.111244	1	3.989263	0.997316
8	20000	4.114686	2.503268	6.617954	1	7.981388	0.997673
12	20000	4.116315	1.669482	5.785797	1	11.96752	0.997293
16	20000	4.105481	1.252983	5.358464	1	15.94559	0.996599
20	20000	4.101935	1.003598	5.105533	1	19.90792	0.995396
24	20000	4.102902	0.839344	4.942246	1	23.80377	0.991824
28	20000	4.099555	0.719843	4.819398	1	27.75543	0.991265
32	20000	4.111426	0.630489	4.741915	1	31.68898	0.990281
36	20000	4.127758	0.561788	4.689546	1	35.56422	0.987895
40	20000	4.123472	0.509048	4.63252	1	39.24886	0.981221
44	20000	4.110007	0.46155	4.571557	1	43.28795	0.983817
48	20000	4.119115	0.425582	4.544697	1	46.94642	0.97805

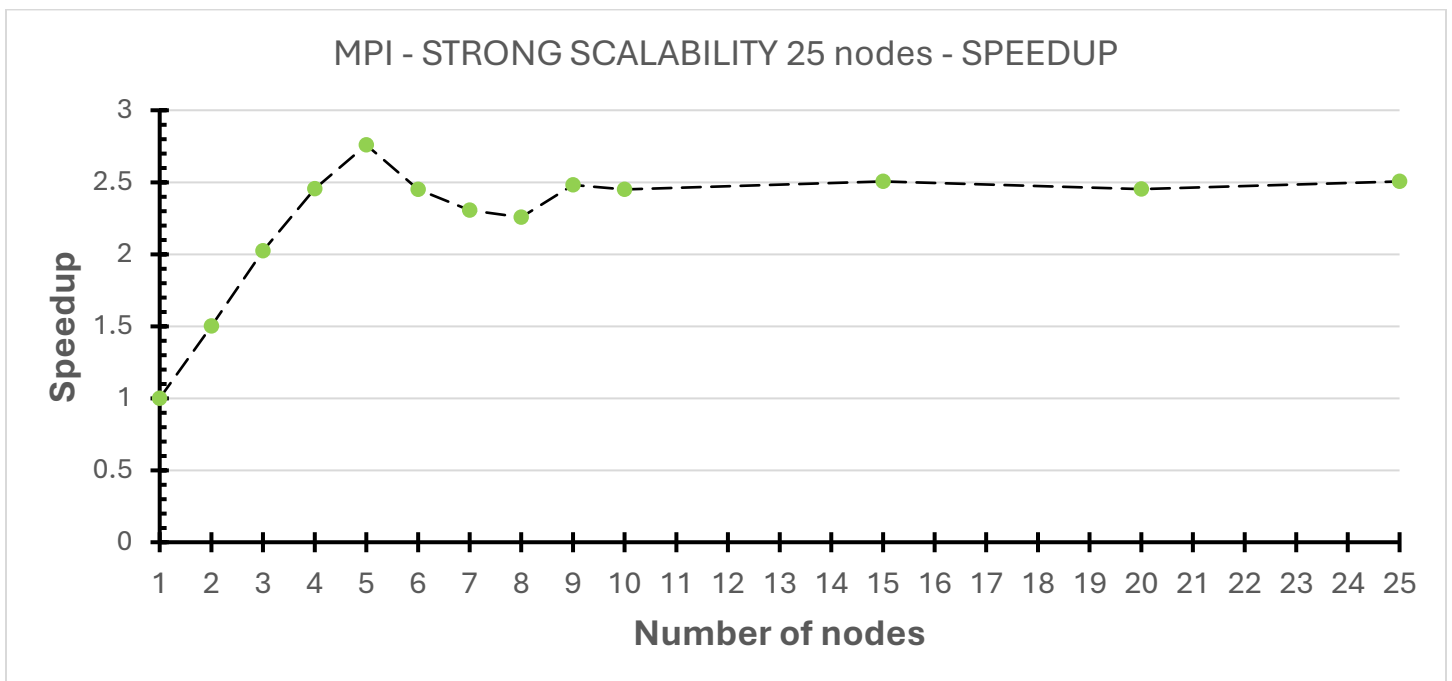
4. Scalabilità weak OMP

WEAK SCALABILITY OMP							
P	N	sequentialT(s)	parallelT(s)	totalT(s)	nodes	Speedup	Efficienza
1	2000	0.039429	0.186552	0.225981	1	1	1
4	4000	0.157891	0.189878	0.347769	1	3.929933958	0.982483489
8	5657	0.319039	0.187418	0.506457	1	7.9630345	0.995379313
12	6928	0.480309	0.190065	0.670374	1	11.77820219	0.981516849
16	8000	0.640143	0.188325	0.828468	1	15.84936679	0.990585424
20	8944	0.800165	0.188221	0.988386	1	19.82265528	0.991132764
24	9798	0.960571	0.191021	1.151592	1	23.438512	0.976604667
28	10583	1.125002	0.191592	1.316594	1	27.2634348	0.9736941
32	11314	1.284721	0.191323	1.476044	1	31.20201962	0.975063113
36	12000	1.446366	0.191788	1.638154	1	35.01716479	0.972699022
40	12649	1.606079	0.192055	1.798134	1	38.85386999	0.97134675
44	13266	1.768412	0.192686	1.961098	1	42.59929626	0.968165824
48	13856	1.929925	0.195418	2.125343	1	45.82226816	0.954630587

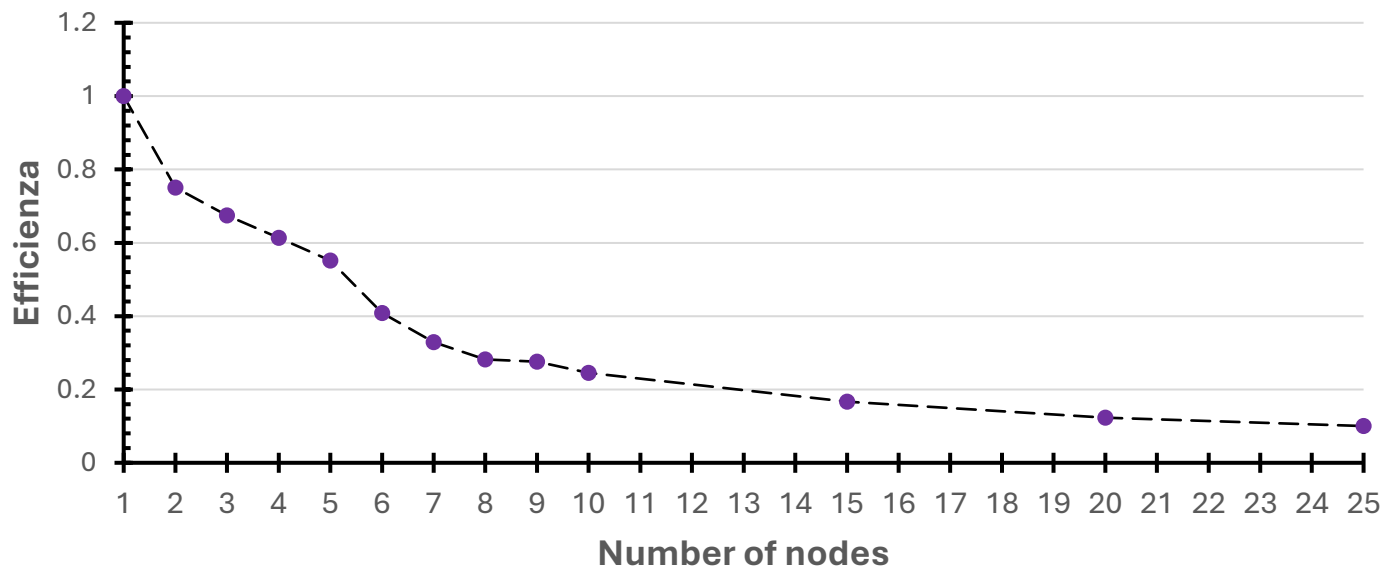
Altre esecuzioni effettuate

1. Scalabilità strong MPI 25 nodi(interessante perché è presente la latenza della rete interna), il grafico di speedup appare identico al caso in cui i processi vengono generati sullo stesso nodo

STRONG SCALABILITY MPI							
P	N	sequentialT(s)	parallelT(s)	totalT(s)	nodes	Speedup	Efficienza
1	20000	7.823305	5.078388	12.901693	1	1	1
2	20000	7.82808	3.382708	11.210788	2	1.501278857	0.750639429
3	20000	7.824769	2.508844	10.333613	3	2.02419441	0.67473147
4	20000	7.825057	2.068172	9.893229	4	2.455495965	0.613873991
5	20000	7.855334	1.840351	9.695685	5	2.75946708	0.551893416
6	20000	7.82041	2.072987	9.893397	6	2.449792497	0.40829875
7	20000	7.823928	2.203233	10.027161	7	2.304970922	0.32928156
8	20000	7.825595	2.249431	10.075026	8	2.257632263	0.282204033
9	20000	7.824626	2.046374	9.871	9	2.481651937	0.275739104
10	20000	7.821708	2.072825	9.894533	10	2.449983959	0.244998396
15	20000	7.815111	2.027055	9.842166	15	2.505303507	0.167020234
20	20000	7.826522	2.070149	9.896671	20	2.453150957	0.122657548
25	20000	7.821016	2.027259	9.848275	25	2.505051402	0.100202056



MPI - STRONG SCALABILITY 25 nodes- EFFICIENZA



Codice C per risolvere i due problemi

Soluzione MPI

```
matrix_transformation_mpi.c:
1. #include <stdio.h>
2. #include <math.h>
3. #include <sys/time.h>
4. #include <stdlib.h>
5. #include <stddef.h>
6. #include <mpi.h>
7. #include <time.h>
8.
9. #define DEFAULT_MATRIX_SIZE 2000
10. #define ENABLE_DEBUG_OUTPUTS 0
11.
12. int main(int argc, char *argv[]){
13.     double
time_paral_begin,time_paral_end,time_seq_begin,time_seq_end,local_elaps,seq_elaps,global_paral_elaps;
14.     int N = DEFAULT_MATRIX_SIZE; // N = matrix rows/cols size
15.
16.     MPI_Init(&argc, &argv);
17.
18.     int P, rank; // P = number of processes, rank = process id
19.     MPI_Comm_size(MPI_COMM_WORLD, &P);
20.     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
21.
22.     //matrix init
23.     double *A = NULL; //data matrix
24.
25.     // scatter/gather vars
26.     int *sendcounts = NULL; //integer array, number of values to send to each processor(including additional
rows("overlapping rows") needed for calculations)
27.     int *displs = NULL; //integer array, Entry i specifies the displacement relative to sendbuf from which to
take the outgoing data to process i
28.     int *valid_rows = NULL; //number of rows that this process needs to calculate(without overlapping rows)
29.     int *start_row_by_process = NULL; //starting row in the original matrix
30.     int *recvcounts = NULL; //for the gatherv, without additional overlapping rows
31.     int *recvddispls = NULL; //for the gatherv, without additional overlapping rows
32.
33.     if(rank == 0){ // input parameters validation
34.         time_seq_begin = MPI_Wtime();
35.
36.         //reading command line parameters
37.         if (argc > 2){
38.             printf("Usage: %s <matrix size>\n", argv[0]);
39.             MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
40.         }else if (argc == 2){
41.             N = atoi(argv[1]);
42.         }
43.
44.         //A initialization with random values
45.         A = malloc(N * N * sizeof(double));
46.         srand((unsigned)time(NULL));
47.         for (int i = 0; i < N * N; i++)
48.             A[i] = rand() % 100;
49.
50.         // A output
51.         if(ENABLE_DEBUG_OUTPUTS){
52.             printf("[proc %d] T:\n", rank);
53.             for(int i=0;i<N;i++){
54.                 for(int j=0;j<N;j++){
55.                     printf("%f\t",A[i*N + j]);
56.                     printf("\n");
57.                 }
58.             }
59.
60.             ////////// --- SCATTER/GATHER preparation---
```

```

62.         //scatter of the rows of A in blocks of N/P rows + 1/2 overlap rows for each block
63.         // sendcounts[r] -> integer array (of length group size) specifying the number of elements to send to
each processor
64.         // displs[r] -> where the data for the rank r process starts
65.         // local_rows -> how many rows the process has
66.         //
67.         // only rank 0 calculate the portions... after he sends the data to the processes
68.         //calculating which rows each process needs
69.         sendcounts = malloc(P * sizeof(int));
70.         displs = malloc(P * sizeof(int));
71.         valid_rows = malloc(P * sizeof(int));
72.         start_row_by_process = malloc(P * sizeof(int));
73.         recvcounts = malloc(P * sizeof(int)); //containing the number of elements that are received from each
process
74.         recvdispls = malloc(P * sizeof(int)); //displacement relative to recvbuf at which to place the
incoming data from process r
75.
76.         int rows_per_node = N / P; // standard number of rows for each process
77.         int remaining_rows = N % P; // needed if there is an uneven number of rows for each process
78.         int recv_disp = 0;
79.
80.         int current = 0;
81.         for (int r = 0; r < P; r++) {
82.             int rows = rows_per_node;
83.             if(r<remaining_rows){ // the first processes get the uneven rows
84.                 rows+=1;
85.             }
86.
87.
88.             int start = current;
89.             int end = current+rows-1;
90.
91.             int send_start = start - 1; //adding one overlap rows at the beginning of the rows for this
92.             if(start == 0){ //first process
93.                 send_start = start;
94.             }
95.
96.             int send_end = end + 1;
97.             if(end == N - 1){ //last process
98.                 send_end = end;
99.             }
100.
101.             valid_rows[r]=rows; //number of rows that this process must output(without overlap rows needed for
doing the calculations)
102.             start_row_by_process[r]=start; //first row that this process must edit
103.
104.             //scatter vars
105.             sendcounts[r]=(send_end-send_start+1)*N; //number of rows * columns for each row
106.             displs[r]=send_start * N; //starting position for the first element for the r process
107.
108.             //gather vars
109.             recvcounts[r]=rows * N;
110.             recvdispls[r]=recv_disp;
111.             recv_disp+=recvcounts[r];
112.
113.             current+=rows;
114.         }
115.         time_seq_end = MPI_Wtime();
116.     }
117.
118.     // parallel logic starts here, initial sync barrier:
119.     MPI_Barrier(MPI_COMM_WORLD);
120.     time_paral_begin=MPI_Wtime();
121.
122.     //sending matrix size to all processes
123.     MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD); // int broadcast from process 0 to all the other processes
124.
125.     if(P > N){
126.         if(rank==0) printf("error: there are more processes( %d ) than the matrix size %d.\n",P, N);
127.         MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
128.     }
129.     //printf("[process %d / %d]\n",rank,size);

```

```

130.
131.
132. //----- SCATTER-----
133.
134. // from sendcounts(array from rank 0) to my_elems_count
135. int my_elems_count;
136. MPI_Scatter(sendcounts, 1, MPI_INT, &my_elems_count, 1, MPI_INT, 0, MPI_COMM_WORLD);
137. int my_rows = my_elems_count / N;
138.
139. int my_start_row;
140. int my_valid_rows;
141. MPI_Scatter(start_row_by_process, 1, MPI_INT,
142.             &my_start_row, 1, MPI_INT,
143.             0, MPI_COMM_WORLD);
144. MPI_Scatter(valid_rows, 1, MPI_INT,
145.             &my_valid_rows, 1, MPI_INT,
146.             0, MPI_COMM_WORLD);
147.
148. // from A(from rank 0) to my_matrix
149. double *my_matrix = malloc(my_elems_count * sizeof(double)); //includes overlaps
150. MPI_Scatterv(A, sendcounts, displs, MPI_DOUBLE,
151.             my_matrix, my_elems_count, MPI_DOUBLE,
152.             0, MPI_COMM_WORLD);
153.
154. //DEBUG: printing sub matrixes
155. if(ENABLE_DEBUG_OUTPUTS){
156.     MPI_Barrier(MPI_COMM_WORLD);
157.     for (int r = 0; r < P; r++){
158.         if (rank == r){
159.             int global_start = my_start_row;
160.             printf("Rank %d rows %d -> %d\n",
161.                 rank,
162.                 global_start,
163.                 global_start + my_rows - 1);
164.
165.             for (int i = 0; i < my_rows; i++){
166.                 printf("Rank %d row %d: ",
167.                     rank, global_start + i);
168.                 for (int j = 0; j < N; j++)
169.                     printf("%6.2f ", my_matrix[i*N + j]);
170.                 printf("\n");
171.             }
172.             printf("\n");
173.         }
174.         MPI_Barrier(MPI_COMM_WORLD);
175.     }
176. }
177.
178. //----- parallel calculations: each node calculate its part of the result matrix-----
179. int *my_ris = malloc(my_valid_rows*N*sizeof(int));
180. int r_without_overlap, offset=0; //to remove the my_matrix beginning overlap if present
181. if(my_start_row!=0){ //if my_matrix contains a overlap row at the beginning i ignore it for calculations
182.     offset=1;
183. }
184.
185. for(int r = 0 ; r<my_valid_rows; r++){
186.     r_without_overlap = r+offset;
187.     for(int c=0;c<N;c++){
188.         // calculating result for element [r][c]
189.         double average = 0;
190.         int elements = 0;
191.         for(int r2 = r_without_overlap-1 ; r2<=r_without_overlap+1; r2++){
192.             for(int c2=c-1;c2<=c+1;c2++){
193.                 if(r2>=0 && r2<my_rows && c2 >= 0 && c2 < N){
194.                     average += my_matrix[r2*N + c2];
195.                     elements++;
196.                 }
197.             }
198.         }
199.         average /= elements;
200.         if(my_matrix[r_without_overlap*N + c] > average) my_ris[r*N + c] = 1;
201.         else my_ris[r*N + c] = 0;

```

```

202.     }
203. }
204.
205. ////////////////////////////////////////////////// --- GATHER---
206.
207. //results gather into rank 0
208. int *T = NULL; //results matrix
209. if (rank == 0){
210.     T = malloc(N * N * sizeof(int));
211. }
212.
213. //MPI_Gatherv: doesnt contain overlap rows
214. MPI_Gatherv(my_ris, my_valid_rows * N, MPI_INT,
215.             T, recvcunts, recvdispls, MPI_INT,
216.             0, MPI_COMM_WORLD);
217.
218. if (rank == 0){
219.     if(ENABLE_DEBUG_OUTPUTS){
220.         printf("\nRESULT MATRIX T:\n");
221.         for (int i = 0; i < N; i++){
222.             for (int j = 0; j < N; j++){
223.                 printf("%d ", T[i*N + j]);
224.             }
225.             printf("\n");
226.         }
227.     }
228. }
229. }
230.
231.
232. MPI_Barrier(MPI_COMM_WORLD);
233. time_paral_end=MPI_Wtime();
234. local_elaps= time_paral_end-time_paral_begin;
235. MPI_Reduce(&local_elaps, &global_paral_elaps,1,MPI_DOUBLE,MPI_MAX,0,MPI_COMM_WORLD);
236. if (rank == 0){
237.     seq_elaps = time_seq_end-time_seq_begin;
238.     //csv format output
239.     //printf("processes;matrix size;seq_elaps;elapsed parallel time\n");
240.     printf("%d;%d;%f;%f;\n",P, N, seq_elaps,global_paral_elaps);
241. }
242.
243. free(my_matrix);
244. free(my_ris);
245. if (rank == 0) {
246.     free(A);
247.     free(sendcounts);
248.     free(displs);
249.     free(T);
250.     free(valid_rows);
251.     free(start_row_by_process);
252.     free(recvcunts);
253.     free(recvdispls);
254. }
255.
256. MPI_Finalize();
257. return EXIT_SUCCESS;
258. }
259.

```

Soluzione OpenMP

```
matrix_transformation_omp.c:
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <omp.h>
4. #include <time.h>
5.
6. #define DEFAULT_MATRIX_SIZE 2000
7.
8. int main(int argc, char* argv[]){
9.     double time_paral_begin,time_paral_end,time_seq_begin,time_seq_end;
10.
11.     time_seq_begin = omp_get_wtime();
12.
13.     //printf("Program started\n");
14.     fflush(stdout);
15.
16.     if (argc!=3){
17.         printf("Usage: %s <n_proc> <matrix size>\n", argv[0]);
18.         exit(1);
19.     }
20.
21.     int P, N, my_rank;
22.     P = atoi(argv[1]); //threads number
23.     N = atoi(argv[2]); //matrix size
24.     if(P > N)
25.     {
26.         printf("error: there are more threads( %d ) than the matrix size %d.\n",P, N);
27.         exit(1);
28.     }
29.
30.
31.
32.     // inizializzazione vettori:
33.
34.     double *A = NULL;
35.     int *R = NULL; // A=input/data matrix, R= results matrix. Both shared between threads
36.     A = malloc(N * N * sizeof(double));
37.     R = malloc(N * N * sizeof(int));
38.
39.     srand((unsigned int)time(NULL));
40.     for(int r=0;r<N;r++){
41.         for(int c=0;c<N;c++){
42.             A[r*N+c]=rand()%100;
43.         }
44.     }
45.
46.     // debug
47.     //printf("[Master] A matrix:\n");
48.     //for(int i=0;i<N;i++){
49.     //    printf("\t%f\n",A[i]);
50.     //}
51.
52.     time_seq_end = omp_get_wtime();
53.     time_paral_begin = omp_get_wtime();
54.
55.     # pragma omp parallel num_threads(P) shared(A,R) private(my_rank)
56.     {
57.         my_rank=omp_get_thread_num();
58.         //printf("thread %d / %d started...\n", my_rank, P);
59.         # pragma omp for
60.         for(int r = 0 ; r<N; r++){
61.             for(int c=0;c<N;c++){
62.
63.                 // calculating result for element [r][c]
64.                 double average = 0;
65.                 int elements = 0;
66.                 for(int r2 = r-1 ; r2<=r+1; r2++){
67.                     for(int c2=c-1;c2<=c+1;c2++){
68.                         if(r2>=0 && r2<N && c2 >= 0 && c2 < N){
```

```

69.                                     average += A[r2*N + c2];
70.                                     elements++;
71.                                     }
72.                                 }
73.                            }
74.                            average /= elements;
75.                            if(A[r*N + c] > average) R[r*N + c] = 1;
76.                            else R[r*N + c] = 0;
77.                        }
78.                    }
79.                }
80.
81.    time_paral_end = omp_get_wtime();
82.    //printf("[Master] result:\n");
83.    //for (int i = 0; i < N; i++){
84.        //    for (int j = 0; j < N; j++){
85.            //        printf("%d ", R[i*N + j]);
86.        //    }
87.        //    printf("\n");
88.    //}
89.
90.    //csv format output
91.    //printf("processes;matrix size;seq_elaps;elapsed parallel time\n");
92.    double seq_elaps = time_seq_end-time_seq_begin;
93.    double paral_elaps = time_paral_end-time_paral_begin;
94.    printf("%d;%d;%f;%f;\n",P, N, seq_elaps,paral_elaps);
95.    free(A);
96.    free(R);
97.    return EXIT_SUCCESS;
98. }
99.

```

Script usati per automatizzare il lancio dei test

Launcher mpi scalabilità strong

```
launcherMpiStrong.sh
1. #!/bin/bash
2.
3. #SBATCH --account=tra25_Infinfo
4. #SBATCH --partition=g100_usr_prod
5. #SBATCH --nodes=1 # Richiediamo il massimo numero di nodi previsti
6. #SBATCH --ntasks-per-node=48 # Massimo numero di task per nodo
7. #SBATCH -o job.out
8. #SBATCH -e job.err
9. #SBATCH --time=00:10:00
10. #SBATCH --mail-type=ALL
11. #SBATCH --mail-user=lorenzo@deluca.pro
12. #SBATCH --job-name=LDLperformanceOptimization
13. module load autoload intelmpi
14. N_SIZE=20000
15.
16. echo "Esecuzione con P=1 (1 Nodo, 1 TPN)"
17. srun -N 1 -n 1 ./matrix_transformation_mpi $N_SIZE
18.
19. for p in $(seq 4 4 48)
20. do
21.     echo "Esecuzione con P=$p (1 Nodo, $p TPN)"
22.     # srun -N $nodes distribuisce i task sui nodi richiesti
23.     srun -N 1 -n $p ./matrix_transformation_mpi $N_SIZE
24. done
25.
26. echo "Test completati. Dati salvati"
27.
```


Launcher mpi scalabilità weak

```
launcherMpiWeak.sh
1. #!/bin/bash
2. #SBATCH --account=tra25_Infinfo
3. #SBATCH --partition=g100_usr_prod
4. #SBATCH --nodes=1
5. #SBATCH --ntasks-per-node=48
6. #SBATCH -o job.out
7. #SBATCH -e job.err
8. #SBATCH --time=00:10:00
9. #SBATCH --mail-type=ALL
10. #SBATCH --mail-user=lorenzo@deluca.pro
11. #SBATCH --job-name=WeakScaling_N
12.
13. module load autoloader intelmpi
14.
15. N_BASE=2000 # righe per processo
16.
17. echo "Inizio test Weak Scaling: righe per processo costanti..."
18.
19. # Numero di processi
20. PROCS=(1 2 3 $(seq 4 4 48) )
21.
22. for p in "${PROCS[@]}"
23. do
24.     # N cresce linearmente con p
25.     N=$(awk -v n_base=$N_BASE -v p=$p 'BEGIN { print int(n_base * sqrt(p) + 0.5) }')
26.
27.     # 1 nodo per tutti i processi, così da togliere la latenza della rete tra i nodi e poter avere un confronto
    più facile con omp
28.     NODES=1
29.
30.     echo "Esecuzione: p=$p, N=$N su $NODES nodi"
31.
32.     srun -N $NODES -n $p ./matrix_transformation_mpi $N
33. done
34.
35. echo "Test completati. Dati salvati"
36.
```

Launcher omp scalabilità strong

```
launcherOmpStrong.sh
1. #!/bin/bash
2.
3. #SBATCH --account=tra25_Infinfo
4. #SBATCH --partition=g100_usr_prod
5. #SBATCH --nodes=1                # Sempre 1 nodo per OpenMP
6. #SBATCH --ntasks=1              # 1 solo processo (che poi lancia i thread)
7. #SBATCH --cpus-per-task=48      # Prenotiamo tutti i 48 core del nodo
8. #SBATCH -o job.out
9. #SBATCH -e job.err
10. #SBATCH --time=00:10:00
11. #SBATCH --job-name=OMP_Strong
12.
13. N_SIZE=20000
14.
15. echo "Inizio test Strong Scaling OpenMP (N=$N_SIZE)..."
16.
17. # Iterazione 1: Caso base con 1 thread
18. echo "Esecuzione con P=1"
19. ./matrix_transformation_omp 1 $N_SIZE
20.
21. # Iterazioni successive: da 4 a 48 a passi di 4
22. for p in $(seq 4 4 48)
23. do
24.     echo "Esecuzione con P=$p"
25.     # Lanciamo il programma passando P come primo argomento
26.     ./matrix_transformation_omp $p $N_SIZE
27. done
28.
29. echo "Test completati. Dati salvati"
30.
```

Launcher omp scalabilità weak

```
launcherOmpWeak.sh
1. #!/bin/bash
2.
3. #SBATCH --account=tra25_Infinfo
4. #SBATCH --partition=g100_usr_prod
5. #SBATCH --nodes=1                # Sempre 1 nodo per OpenMP
6. #SBATCH --ntasks=1              # 1 solo processo (che poi lancia i thread)
7. #SBATCH --cpus-per-task=48      # Prenotiamo tutti i 48 core del nodo
8. #SBATCH -o job.out
9. #SBATCH -e job.err
10. #SBATCH --time=00:10:00
11. #SBATCH --job-name=OMP_Weak
12.
13. N_BASE=2000    # righe per processo
14.
15. echo "Inizio test weak Scaling OpenMP (N=$N_SIZE)..."
16.
17. # Iterazione 1: Caso base con 1 thread
18. echo "Esecuzione con P=1"
19. ./matrix_transformation_omp 1 $N_BASE
20.
21. # Iterazioni successive: da 4 a 48 a passi di 4
22. for p in $(seq 4 4 48)
23. do
24.     # Calculate N = N_BASE * sqrt(p)
25.     N=$(awk -v n_base=$N_BASE -v p=$p 'BEGIN { print int(n_base * sqrt(p) + 0.5) }')
26.
27.     echo "Esecuzione: P=$p, N=$N su 1 nodo (tpn=1)"
28.     # Lanciamo il programma passando P come primo argomento
29.     ./matrix_transformation_omp $p $N
30. done
31.
32. echo "Test completati. Dati salvati"
33.
```